

Mastering Sliding window concepts

Longest Repeating Character Replacement & Permutation in String

Presented by: QMML Society

Problem Statement

Problem: You are given a string `s` of uppercase English characters and an integer `k`. You can change up to `k` characters to ANY other character. Return the **longest valid substring** of identical characters.

Examples

- **Example 1:** `s = "XYXX", k = 2`
 - Output: `4` (Replace the two 'X's with 'Y's -> "YYYY")
- **Example 2:** `s = "AAABABB", k = 1`
 - Output: `5` (Replace the one 'B' in "AAABA" to get "AAAAA")

Conceptual Intuition: The Validity Equation

How do we actually know if a substring is "valid"?

We don't need to actually *change* characters in the string! We just need mathematical proof that a replacement *could* be done.

Within any window (**left** to **right**):

1. **Total length** of the window: **$\text{right} - \text{left} + 1$**
2. **Most frequent character count**: The max occurrences of any single letter in that window.
3. **Number of characters to replace**:

$$\text{Total Length} - \text{Most Frequent Count}$$

The Golden Rule:

A window is valid strictly if: **$(\text{Length} - \text{MaxFreq}) \leq k$**

Example: If **$s = \text{"AAB"}$** and **$k = 1$** . In the window **$[\text{A A B}]$** , $\text{Length} = 3$, $\text{MaxFreq} = 2$ (for 'A').

$3 - 2 = 1 \leq 1$. This window is valid!

Brute Force vs. Sliding Window

✗ Brute Force ($O(n^2 \cdot 26)$)

- Check every possible substring $O(n^2)$.
- Count the frequency of all 26 characters in that substring $O(26)$.
- Too slow!



✓ Sliding Window ($O(n)$)

- Keep an expanding window `[left, right]`.
- Maintain a fast frequency map (or `int[26]`) of the active window.
- When `(Length - MaxFreq) > k`, the window is mathematically **invalid**.
- To fix it, simply increment `left` to shrink the window until valid again!

Step-by-Step Visualization

Imagine `s = "AABABBA"`, `k = 1`

```
1. [A] A B A B B A    -> Freq: A:1. Max: 1. (1 - 1 = 0 <= 1) ✓ MaxLen=1
2. [A A] B A B B A    -> Freq: A:2. Max: 2. (2 - 2 = 0 <= 1) ✓ MaxLen=2
3. [A A B] A B B A    -> Freq: A:2, B:1. (3 - 2 = 1 <= 1) ✓ MaxLen=3
4. [A A B A] B B A    -> Freq: A:3, B:1. (4 - 3 = 1 <= 1) ✓ MaxLen=4
5. [A A B A B] B A    -> Freq: A:3, B:2. (5 - 3 = 2 > 1) ✗ INVALID!
   (Shrink Left)
   A [A B A B] B A    -> Freq: A:2, B:2. Max: 2. (4 - 2 = 2 > 1) ✗ INVALID!
   (Shrink Left)
   A A [B A B] B A    -> Freq: A:1, B:2. Max: 2. (3 - 2 = 1 <= 1) ✓ Valid.
6. A A [B A B B] A    -> Freq: A:1, B:3. Max: 3. (4 - 3 = 1 <= 1) ✓ MaxLen=4
```

Sliding Window Logic & Pseudocode

```
function characterReplacement(string s, int k):
    left = 0
    max_frequent_count = 0
    max_length = 0
    counts = array of size 26 (all zeros)

    for right from 0 to length(s) - 1:
        char_index = ascii(s[right]) - ascii('A')
        counts[char_index]++

        max_frequent_count = maximum(max_frequent_count, counts[char_index])

        // Is the current window mathematically invalid?
        while (right - left + 1) - max_frequent_count > k:
            left_char_index = ascii(s[left]) - ascii('A')
            counts[left_char_index]--
            left++

        // Window is valid!
        max_length = maximum(max_length, right - left + 1)

    return max_length
```

Optimal Java Implementation

```
class Solution {
    public int characterReplacement(String s, int k) {
        int[] counts = new int[26];
        int left = 0;
        int maxFreq = 0;
        int maxLen = 0;

        for (int right = 0; right < s.length(); right++) {
            int rightIdx = s.charAt(right) - 'A';
            counts[rightIdx]++;
            maxFreq = Math.max(maxFreq, counts[rightIdx]);

            // If window is invalid, explicitly shrink from the left
            while ((right - left + 1) - maxFreq > k) {
                int leftIdx = s.charAt(left) - 'A';
                counts[leftIdx]--;
                left++;
            }
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```

Optimal Python Implementation

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        counts = {}
        left = 0
        max_freq = 0
        max_len = 0

        for right in range(len(s)):
            counts[s[right]] = counts.get(s[right], 0) + 1
            max_freq = max(max_freq, counts[s[right]])

            # If (total elements) - (most frequent element) > k, invalid window!
            while (right - left + 1) - max_freq > k:
                counts[s[left]] -= 1
                left += 1

            max_len = max(max_len, right - left + 1)

        return max_len
```


$O(N)$ Optimization Nuance

You might notice that when we shrink the window by moving `left`, we **decrement** `counts[s[left]]` but we do NOT re-scan the array to update `max_frequent_count` or `max_freq` to a smaller value!

Why is this perfectly mathematically safe?

Because we are exclusively looking for the **Longest** substring.

Our `max_length` can *only* increase if we find a window that supports a historically *larger* `max_freq`.

A smaller `max_freq` could never physically yield a larger window size than what we've previously recorded! Thus, ignoring the deflation of `max_freq` saves us an $O(26)$ scan inside the loop, guaranteeing an absolutely strict $O(N)$ runtime!

Permutation in String

Problem: This problem asks us to determine if **s2** contains any substring that is a permutation of **s1**. Both strings consist only of lowercase English letters.

This is a classic problem that can be efficiently solved using the **Sliding Window** technique combined with **Frequency Maps/Arrays**.

Core Concepts for Java & Python Students

Before diving into the solution, let's review some fundamental concepts relevant to string manipulation and problem-solving in both Java and Python.

Strings

- **Definition:** A sequence of characters.
- **Immutability:**
 - **Java:** `String` objects are immutable. Any operation that seems to modify a string (like `substring()`) actually creates a new string object.
 - **Python:** Strings (`str` type) are also immutable. Similar to Java, operations like slicing create new string objects.
- **Length:**
 - **Java:** `s.length()` method.
 - **Python:** `len(s)` function.

Core Concepts: Access & Substrings

Character Access:

- **Java:** `s.charAt(index)` returns the character at a specific index.
- **Python:** `s[index]` accesses the character at a specific index.
 - *Example:* For `s = "cat"`, index 1 is `'a'`.

Substrings/Slicing:

- **Java:** `s.substring(startIndex, endIndex)` returns a new string from `startIndex` (inclusive) up to `endIndex` (exclusive).
- **Python:** `s[startIndex:endIndex]` creates a slice (substring) from `startIndex` (inclusive) up to `endIndex` (exclusive).
 - *Example:* For `s = "hello"`, from index 1 to 4 is `"ello"`.

Permutations

- **Definition:** A permutation of a string is another string formed by rearranging its letters. For example, "abc", "bac", "cab", "bca", "cba", "acb" are all permutations.
- **Key Property:** Two strings are permutations of each other if and only if they have the same characters with the *exact same frequencies*. For example, "aabbcc" and "bcaba" are permutations (two 'a's, two 'b's, one 'c').
- **Checking Permutations:** The most efficient way to check if two strings are permutations is to count the frequency of each character in both strings and compare these frequency counts.

Substrings & Frequency Arrays

Substrings

- **Definition:** A contiguous sequence of characters within a string. ("car" is a substring of "cardiac", but "cad" is not).

Frequency Maps / Arrays

- **Concept:** A data structure used to store the count of occurrences for each item.
- **Implementation:** Since there are only 26 lowercase English letters, we can use an array of size 26 (`int[26]` in Java, `list` in Python) where each index corresponds to a letter (0 for 'a', 1 for 'b', 25 for 'z').
 - **Index:** `c - 'a'` in Java, `ord(c) - ord('a')` in Python.
 - Provide $O(1)$ access and update.

Brute-Force Approach

A straightforward, but highly inefficient, approach would be to generate all possible permutations of `s1` and then check if any of these permutations exist as a substring within `s2`.

Idea

1. Generate every possible permutation of the string `s1`.
2. For each generated permutation, check if it exists as a substring in `s2`.
3. If we find any permutation that is a substring of `s2`, return true.
4. If we iterate through all permutations and none are found in `s2`, return false.

Brute-Force: Time & Space Complexity

Time Complexity

- **Generating Permutations:** For a string of length L_1 , there are $L_1!$ permutations. Generating each takes $O(L_1)$ time. ($O(L_1! \cdot L_1)$)
- **Checking Substring:** For each generated permutation, checking if it's a substring of **s2** (length L_2) can take up to $O(L_1 \cdot L_2)$.
- **Total Time Complexity:** $O(L_1! \cdot L_1 \cdot L_2)$.
Given that L_1 can be up to 10^4 , $L_1!$ grows extremely rapidly. This approach is only feasible for very small **s1** lengths.

Space Complexity

- **Storing Permutations:** We would need to store the generated permutations. In the worst case, $O(L_1! \cdot L_1)$ space.

(This method is generally not acceptable due to its exponential time complexity).

Optimal Code Solutions

The optimal approach uses a **Sliding Window** technique combined with **Frequency Arrays**.

Instead of generating permutations, we realize that any substring of **s2** that is a permutation of **s1** must have the same length as **s1** and the same character frequencies as **s1**.

Approach: Sliding Window Setup

1. **Initial Check:** If `s1` is longer than `s2`, it's impossible for `s2` to contain a permutation of `s1`. Return false.
2. **Frequency Arrays Setup:** Create two frequency arrays, `s1_freq` and `window_freq`, each of size 26 (for 'a' through 'z').
3. **Populate `s1_freq`:** Iterate through `s1` and populate `s1_freq` with the counts of its characters.
4. **Initialize the First Window:** Iterate through the first `len(s1)` characters of `s2` and populate `window_freq` with their counts.
5. **Check Initial Window:** Compare `s1_freq` and `window_freq`. If they are identical, return true.

Approach: Sliding Window Execution

Slide the Window:

Now, we slide our window across `s2`. The window will always have the length of `s1`.

Starting from index = `len(s1)` up to `len(s2) - 1`:

1. **Add new character:** Take the character `s2[index]` (entering the window on the right) and increment its count in `window_freq`.
2. **Remove old character:** Take the character `s2[index - len(s1)]` (leaving the window on the left) and decrement its count in `window_freq`.
3. **Check for match:** After updating `window_freq`, compare it with `s1_freq`. If they are identical, return true.

No Match Found: If the loop finishes without finding any matching permutations, return false.

Helper Function: `matches()`

To compare the two frequency arrays efficiently, we can use a helper function `matches(arr1, arr2)` that iterates through all 26 possible character counts.

If all counts are equal, it returns true; otherwise, false. This comparison takes $O(1)$ time because the array size is fixed (26).

(In Python, we can simply compare the lists directly: `s1_freq == window_freq`)

Java Solution

```
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        if (s1.length() > s2.length()) return false;

        int[] s1_freq = new int[26];
        int[] window_freq = new int[26];

        for (int i = 0; i < s1.length(); i++) {
            s1_freq[s1.charAt(i) - 'a']++;
            window_freq[s2.charAt(i) - 'a']++;
        }

        if (matches(s1_freq, window_freq)) return true;

        for (int right = s1.length(); right < s2.length(); right++) {
            window_freq[s2.charAt(right) - 'a']++;
            window_freq[s2.charAt(right - s1.length()) - 'a']--;

            if (matches(s1_freq, window_freq)) return true;
        }
        return false;
    }

    private boolean matches(int[] arr1, int[] arr2) {
        for (int i = 0; i < 26; i++) {
            if (arr1[i] != arr2[i]) return false;
        }
        return true;
    }
}
```

Python Solution

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        if len(s1) > len(s2): return False

        s1_freq = [0] * 26
        window_freq = [0] * 26

        for char in s1:
            s1_freq[ord(char) - ord('a')] += 1

        for i in range(len(s1)):
            window_freq[ord(s2[i]) - ord('a')] += 1

        if s1_freq == window_freq: return True

        for right in range(len(s1), len(s2)):
            window_freq[ord(s2[right]) - ord('a')] += 1
            window_freq[ord(s2[right - len(s1)]) - ord('a')] -= 1

            if s1_freq == window_freq: return True

        return False
```

Complexity Analysis (Optimal)

Let L_1 be the length of `s1` and L_2 be the length of `s2`.

Time Complexity

- **Populating Arrays:** Initializing frequencies takes $O(L_1)$ time.
- **Initial comparison:** Comparing 26 elements takes $O(1)$ time.
- **Sliding the window:** The loop runs $L_2 - L_1$ times. Updating `window_freq` takes $O(1)$, and comparing takes $O(1)$. Total for sliding: $O(L_2 - L_1)$.
- **Overall Time Complexity:** $O(L_1 + L_2)$

Space Complexity

- We use two frequency arrays (`s1_freq` and `window_freq`), each of size 26. Since the size 26 is constant and does not depend on the input string lengths, the space used is fixed.
- **Overall Space Complex---

Minimum Window Substring

🧩 1. Background Understanding

This problem is about finding a **smallest substring** in **s** that contains all characters of **t** (with frequency).

Key requirements & Intuition

- Order **does NOT matter**, but Frequency **DOES matter**. Substring must be **contiguous**.
- 🖱️ Find the **smallest “window”** in **s** that satisfies all requirements of **t**

Example Insight

```
s = "OUZODYXAZV", t = "XYZ"
```

You must include: X once, Y once, Z once.

Possible windows: **"ODYXAZ"** ✅, **"YXAZ"** ✅ (smaller → best)

Minimum Window Substring

2. Brute Force Solution

Idea & Steps:

Check **every substring of** `s`, and verify if it contains all characters of `t`.

1. Generate all substrings `s[i:j]`
2. For each substring: Count characters & compare with `t`'s frequency.
3. Track the smallest valid substring.

Minimum Window Substring

Brute Force: Complexity & Pseudocode

Complexity & Why it's bad:

- Substrings: $O(n^2)$, Checking validity: $O(n)$  **Total: $O(n^3)$**  too slow
- Repeated work and doesn't scale for large n .

Pseudocode

```
min_length = infinity, result = ""
for i from 0 to n-1:
    for j from i to n-1:
        if s[i:j] contains all chars of t and length < min_length:
            update result
```

Minimum Window Substring

⚡ 3. Optimal Solution (Sliding Window)

💡 Core Idea & Strategy:

Instead of checking all substrings, **dynamically maintain a valid window**.

- **Expand** window → include characters. When valid → **shrink** window to minimize it.

🧠 Key Variables (Simplified):

- **targetCounts** : How many times we need each character from **t**
- **currentCounts** : How many times each character appears in our current sliding window
- **matchedUniqueChars** : How many distinct characters have met their target frequency
- **requiredUniqueChars** : Total distinct characters we need to match from **t**

Minimum Window Substring

Optimal Solution: Algorithm Steps

Algorithm Steps:

1. Build `targetCounts`. Use two pointers: `left`, `right`.
2. **Expand** `right`: Add character to `currentCounts`. If target is met → increase `matchedUniqueChars`.
3. **When** `matchedUniqueChars == requiredUniqueChars`: Record window length. Try shrinking from `left`. Repeat.

Complexity:

- **Time:** $O(n)$ | **Space:** $O(m)$ (where m is unique characters int)



4. Language-Independent Pseudocode (Array-Optimized)

```
targetCounts = array of size 128 (initialized to 0)
requiredUniqueChars = 0

for char in t:
    if targetCounts[char] == 0: requiredUniqueChars += 1
    targetCounts[char] += 1

currentCounts = array of size 128 (initialized to 0)
matchedUniqueChars = 0
left = 0, minLen = infinity, bestStart = 0

for right from 0 to len(s)-1:
    c = s[right]
    currentCounts[c] += 1

    if targetCounts[c] > 0 AND currentCounts[c] == targetCounts[c]:
        matchedUniqueChars += 1

    while matchedUniqueChars == requiredUniqueChars:
        record new minLen and bestStart if smaller window found

        l_char = s[left]
        currentCounts[l_char] -= 1
        if targetCounts[l_char] > 0 AND currentCounts[l_char] < targetCounts[l_char]:
            matchedUniqueChars -= 1
        left += 1

return s[bestStart to bestStart + minLen]
```



5. Java Solution (Array-Optimized)

```
class Solution {
    public String minWindow(String s, String t) {
        if (s.length() < t.length()) return "";

        int[] targetCounts = new int[128];
        int requiredUniqueChars = 0;
        for (char c : t.toCharArray()) {
            if (targetCounts[c] == 0) requiredUniqueChars++;
            targetCounts[c]++;
        }

        int[] currentCounts = new int[128];
        int matchedUniqueChars = 0;
        int left = 0, minLen = Integer.MAX_VALUE, bestStart = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            currentCounts[c]++;

            if (targetCounts[c] > 0 && currentCounts[c] == targetCounts[c]) {
                matchedUniqueChars++;
            }

            while (matchedUniqueChars == requiredUniqueChars) {
                if ((right - left + 1) < minLen) {
                    minLen = right - left + 1;
                    bestStart = left;
                }
                char leftChar = s.charAt(left);
                currentCounts[leftChar]--;

                if (targetCounts[leftChar] > 0 && currentCounts[leftChar] < targetCounts[leftChar]) {
                    matchedUniqueChars--;
                }
                left++;
            }
        }
        return minLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + minLen);
    }
}
```



6. Python Solution (Array-Optimized)

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if len(s) < len(t): return ""

        target_counts = [0] * 128
        required_unique_chars = 0
        for char in t:
            if target_counts[ord(char)] == 0:
                required_unique_chars += 1
            target_counts[ord(char)] += 1

        current_counts = [0] * 128
        matched_unique_chars = 0

        left = 0
        best_bounds = [-1, -1]
        min_len = float("inf")

        for right in range(len(s)):
            char = s[right]
            c_idx = ord(char)
            current_counts[c_idx] += 1

            if target_counts[c_idx] > 0 and current_counts[c_idx] == target_counts[c_idx]:
                matched_unique_chars += 1

            while matched_unique_chars == required_unique_chars:
                if (right - left + 1) < min_len:
                    best_bounds = [left, right]
                    min_len = right - left + 1

                left_char = s[left]
                l_idx = ord(left_char)
                current_counts[l_idx] -= 1

                if target_counts[l_idx] > 0 and current_counts[l_idx] < target_counts[l_idx]:
                    matched_unique_chars -= 1
                left += 1

        l, r = best_bounds
        return s[l:r+1] if min_len != float("inf") else ""
```

Minimum Window Substring

7. Mental Model & Pattern Recognition

Mental Model (Very Important)

Think in terms of:

-  **Expand** → “Can I satisfy the requirement?”
-  **Shrink** → “Can I make it smaller?”

32

8. When You See This Pattern

Use sliding window when:

- Substring problems
- "Minimum / maximum window"
- Frequency constraints
- Contiguous segments

Sliding Window Maximum

🧩 1. Background Understanding & Problem

Problem: You are given an array `nums` and a fixed window size `k`. The window slides from left → right, one step at a time. At each step, output the **maximum element inside the window**.

🔍 Example Insight

`nums = [1,2,1,0,4,2,6]` , `k = 3`

Windows:

- `[1,2,1] → 2` , `[2,1,0] → 2` , `[1,0,4] → 4` , `[0,4,2] → 4` , `[4,2,6] → 6`

Output: `[2,2,4,4,6]`

Toolbox: The Heap (Priority Queue)

A **Heap** is a specialized tree-based data structure that satisfies the **heap property**:

- **Max-Heap**: The value of each node is $\leq$ the value of its parent ($Root = Max$).
- **Min-Heap**: The value of each node is $\geq$ the value of its parent ($Root = Min$).

Why use it?

Used to implement **Priority Queues** — where you need immediate access to the "most important" (largest or smallest) element.

- **Java**: `PriorityQueue<T>`
- **Python**: `heapq` module (Min-Heap by default)

Heap: Core Operations & Complexity

Heaps are efficient because they maintain a **complete binary tree** structure.

Operation	Action	Complexity
Push	Add element	$O(\log N)$
Pop	Remove root	$O(\log N)$
Peek	View root	$O(1)$

👉 **Note:** Heaps are **not** sorted. Only the root is guaranteed Max/Min.



Heap: The "Lazy Removal" Strategy

In Sliding Window problems, elements often fall out of the window while they are still buried deep inside the heap.

The Problem: Removing an arbitrary element from a binary heap is slow ($O(N)$).

The Solution: Lazy Removal

1. Don't remove "expired" elements if they are buried.
2. Wait until the expired element reaches the **top** (Peek).
3. **Pop** it only then!

👉 This keeps our sliding window moving at $O(\log K)$ per step!

Sliding Window Maximum

2. Brute Force Solution

Idea & Steps:

For every window, scan all **k** elements and find the max.

1. Loop through array
2. For each index **i**, compute max of **nums[i:i+k]**

Complexity:

- Each window: **O(k)**, Total windows: **O(n)**  **Total: O(n * k)** **✗** too slow for large inputs

Pseudocode (Brute Force)

```
result = []
for i from 0 to n-k:
    max_val = -∞
    for j from i to i+k-1:
        max_val = max(max_val, nums[j])
    add max_val to result
return result
```

Sliding Window Maximum

🧩 Sliding Window + Max Heap Approach

Note: The **deque solution** = optimal ($O(n)$). The **heap solution** = $O(n \log k)$.

💡 Idea & ⚠️ Problem:

- Maintain a **max heap** of elements in the current window.
- Heaps don't support efficient arbitrary removal. 🖱️ **Solution: Lazy removal**
- Store **(value, index)**. When top element is out of window (**index <= i - k**) → discard it!

🔄 Algorithm Steps:

For each index **i** :

1. Push **(nums[i], i)** into max heap
2. Remove elements from heap top if: **index <= i - k**
3. If window is valid (**i >= k-1**): Add heap top to result

🕒 Complexity & ⚖️ Heap vs Deque:

- **Time:** $O(n \log k)$ | **Space:** $O(k)$
- **Heap vs Deque:** Heap is $O(n \log k)$ but easier to quickly implement. Deque is $O(n)$ and optimal. Use Heap when you

Java Solution (Heap-Based)

```
import java.util.*;

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        // Max-PriorityQueue storing [value, index]
        PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[0] - a[0]);
        int[] result = new int[nums.length - k + 1];
        int idx = 0;

        for (int i = 0; i < nums.length; i++) {
            // 1. Sliding Window: Expand by adding current element
            maxHeap.offer(new int[]{nums[i], i});

            // 2. Heap Fundamentals: Lazy Removal
            // Remove top elements if they are outside the sliding window
            while (maxHeap.peek()[1] <= i - k) {
                maxHeap.poll();
            }

            // 3. Record result once window size 'k' is reached
            if (i >= k - 1) {
                result[idx++] = maxHeap.peek()[0];
            }
        }
        return result;
    }
}
```

Python Solution (Heap-Based)

```
import heapq

class Solution:
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        # Max-Heap using negative values, storing (-value, index)
        heap, result = [], []

        for i in range(len(nums)):
            # 1. Sliding Window: Push current element onto heap
            heapq.heappush(heap, (-nums[i], i))

            # 2. Heap Fundamentals: Lazy Removal
            # If the index of the max element is out of window, pop it
            while heap[0][1] <= i - k:
                heapq.heappop(heap)

            # 3. Append max to result once window is full
            if i >= k - 1:
                result.append(-heap[0][0])

        return result
```